

Week 5

1. UNIX Process Control (I)

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2560 – System Programming [2025F]

Monday, September 29th, 2025



University of Windsor

Content

- 1 Introduction
- 2 Unix Processes
- 3 Creating a new Process: `fork()`
- 4 Terminating a process: `exit()`
- 5 `wait()` and `waitpid()`

Assignment #1: Overview

Part I

Introduction



University of Windsor

Context

Context

- single computer (single CPU)
- under UNIX operating system

It is possible to have several simultaneously executing programs

→ It appears that the executions are done in Parallel!

Multiprogramming:

The CPU executes some instructions of one program **then switches** to another program giving the **illusion** that any program is continuously executing, this scheme is called **time-slicing**.

Part I

UNIX Processes



Creating a new Process: `fork()`Terminating a process: `exit()``wait()` and `waitpid()`

Program vs. process

A *program* is an executable file residing on a disk.

A *process* is an executing (running) program, usually with a limited life-time.

Note that sometimes a process is called *task*.

Important notion

An executing program → a *process*.

Unix Processes

Every process in Unix contains the following (Chapter 7, optional reading):

1. A unique Process ID (**PID**): A unique nonnegative integer assigned by Unix, used to identify a process.
2. The **User ID** of the owner.
3. Some **code**: Instructions that are being executed (the code segment)
4. Some **data**: Variables (the data segment)
5. A **stack**: A form of memory where it is possible to push and pop instructions (the stack segment).
6. An **environment**: Registers' contents, tables of open files,...

`getpid()` and `getppid()`

`getpid()` and `getppid()`

A process can obtain its **PID** by the `getpid()` system call. A process can also obtain its parent PID **PPID** by the `getppid()` system call.

`//getpid.c`

```
#include <stdio.h>
```

```
int main(void){
```

```
    printf("Hello, my PID is %d\n", getpid());
```

```
    printf("Hello, my PPID is %d\n", getppid());
```

```
    exit(0);
```

```
}
```

```
Shell-Prompt> a.out
```

```
Hello, my PID is 11723
```

```
Hello, my PPID is 5598
```


States of a process

Different states of a process

Throughout its lifetime, a process can have the following states:

New: the process has just been created.

Ready: waiting to be assigned to a processor (the processor is busy running other processes).

Running: while its instructions are being executed.

Blocked: waiting for an event (e.g. I/O).

Done: finished

A new process becomes ready and waits for its turn to be executed by the processor.

A ready process becomes running for a "time slice" after which it is **either done, blocked or ready** to continue for another time slice.

A blocked process becomes ready again after the event it was waiting for occurs (e.g. I/O).

Creating a new Process: `fork()`Terminating a process: `exit()``wait()` and `waitpid()`

init

init

Unix starts as a single process, called `init`. The PID of `init` is 1.

The only way to create a new process in Unix, is to duplicate an existing one.

→ the process `init` is the ancestor of all subsequent processes.

In particular, process `init` never dies.

Creation of a new process

The creation or spawning of new processes is done with two system calls:

`fork()` : duplicates the caller process

`exec()` : replaces the caller process by a new one.

Example 1: Handling a login session

`init` forks a copy of itself.

The child process `execs` the `getty` program which prints the login prompt, waits for the user's login name, and then `execs` the `login` program.

the `login` process, if successful, `execs` the shell program (e.g. `csh`).

Example 2: Running a utility from a shell (`cs`h).

`cs`h forks a copy of itself.

the child process `execs` the utility program, for example `gedit`.

the parent process waits for the termination (`exit`) of its child process by going asleep.

when the child process terminates, a signal is sent to the parent process (the shell program `cs`h). The latter wakes-up and becomes ready to accept the next command.

Part III

Creating a new process: `fork()`



Creating a new Process: `fork()`

`fork()`

Synopsis: `pid_t fork(void);`

When successful, the `fork()` system call:

1. creates a copy of the caller (parent) process.
2. returns the `PID` of the newly created process to the parent
3. returns 0 to the new process (the child).

If not successful, the `fork()` returns -1.

`fork()` is a "**strange**" system call: called by a single process but returns twice, to two different processes.

Creating a new Process: `fork()`Terminating a process: `exit()``wait()` and `waitpid()`

The child process

A child process has:

1. its own unique `PID`,
2. a different `PPID`,
3. its own copy of the parent's data segment and **file descriptors**.

Both parent and children resume the execution after the call to `fork()`.

`fork()` is primarily used in two situations:

- 1 A process wants to execute another program.
- 2 A process has a main task and when necessary, creates a child to handle an operation (Servers).

both parent and children resume the execution after the call to `fork()`

```
//fork1.c
```

```
#include <unistd.h>
```

```
int main(int argc, char *argv[]){
```

```
    int pid;
```

```
    printf("Only one process\n");
```

```
    pid = fork();
```

```
    if(pid == -1){
```

```
        perror("impossible to fork");
```

```
        exit(1);
```

```
    }
```

```
    if(pid > 0)
```

```
        printf("I am the parent, pid=%d\n", getpid());
```

```
    else
```

```
        if(pid == 0)
```

```
            printf("I am the child, pid=%d\n", getpid());
```

```
    exit(0);
```

```
}
```

The child has its own copy of the parent's data segment

```
//fork2.c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int glb = 100; // global variable
int main(){
    int pid;
    int var = 88;
    printf("Before fork\n");
    pid = fork();
    if( pid < 0 ){
        perror("fork");
        exit(1);
    }
    if(pid == 0){ //child
        glb++; var++;
    }
    else{ // parent
        sleep(2);
    }
    printf("pid = %d, glob = %d, var = %d\n", getpid(), glb, var);
    return 0;
}
```

Creating a new Process: `fork()`Terminating a process: `exit()``wait()` and `waitpid()`

The child has its own copy of the parent's file descriptors and share the same offset (`fork3.c`)

Homework →

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

#include <stdlib.h>

int main(){
    int pid, fd, i;
    char c;

    if ( (fd = open("test", O_RDWR | O_CREAT, 0644)) == -1 ){
        perror("test");
    }

    if ( (pid = fork()) < 0 ){
        perror("fork");
        exit(1);
    }

    if (pid == 0) // child
        for ( i = 65; i < 85; i++){ c
            = i; write(fd, &c, 1);
        }
    else // parent
        for ( i = 0 ; i < 20; i++){ c
            = 64; // character @
            write(fd, &c, 1);
        }
    return 0;
}
```

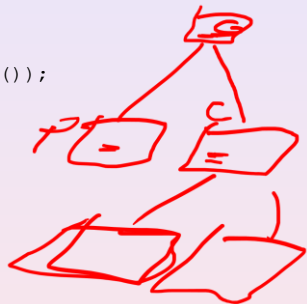
What is the output?

//q1.c

```
int main(){
    printf("before fork, my pid is %d\n" , getpid());
    fork();
    fork();
    fork();
    printf("done, my pid is %d\n", getpid());
}
```

//q2.c

```
int main(){
    int i;
    printf("before fork, my pid is %d\n" , getpid());
    for (i=0; i<3; i++){
        if ( fork()== 0 )
            printf("Hi, I am child. My pid is %d\n", getpid());
    }
}
```



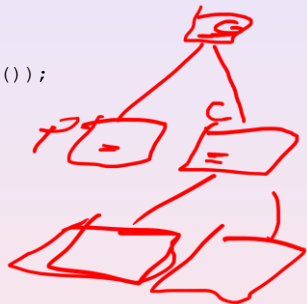
What is the output?

```
//q1.c
```

```
int main(){
    printf("before fork, my pid is %d\n" , getpid());
    fork();
    fork();
    fork();
    printf("done, my pid is %d\n", getpid());
}
P0
```

```
//q2.c
```

```
int main(){
    int i;
    printf("before fork, my pid is %d\n" , getpid());
    for (i=0; i<3; i++){
        if (fork() == 0 )
            printf("Hi, I am child. My pid is %d\n", getpid());
    }
}
P1
P3
P7
P5
P2
P6
P4
```



Terminating a process: `exit()`

`wait()` and `waitpid()`

What is the output?

```
int main(){
    printf("before fork, my pid is %d\n" , getpid());
    fork();
    fork();
    fork();
    printf("done, my pid is %d\n", getpid());
}
```

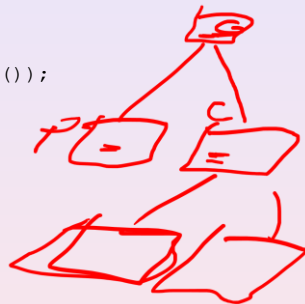
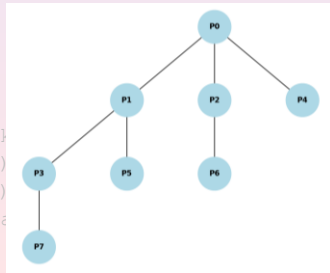
P0

// g2.c P1

```

int main() {
    P3
    int i;
    P7
    printf("before fork\n");
    P5
    for (i=0; i<3; i++)
    {
        P2
        if (fork() == 0)
        {
            P6 ("Hi, I am child\n");
        }
        P4
    }
}

```



What is the output?

```
//q1.c
```

```
int main(){  
    printf("before fork, my pid is %d\n" , getpid());  
    fork();  
    fork();  
    fork();  
    printf("done, my pid is %d\n", getpid());  
}
```

```
//q2.c
```

```
int main(){  
    int i;  
    printf("before fork, my pid is %d\n" , getpid());  
    for (i=0; i<3; i++){  
        if ( fork()== 0 )  
            printf("Hi, I am child. My pid is %d\n", getpid());  
    }  
}
```



What is the output?

P0: `q2.c`

```
int main() {
    P1
    print("before fork, my pid is %d\n", getpid());
    P3
    fork(); P7
    P5
    fork();
    P2
    fork();
    P6
    P4
    printf("done, my pid is %d\n", getpid());
}
```

//q2.c

```
int main() {
    int i;
    printf("before fork, my pid is %d\n", getpid());
    for (i=0; i<3; i++) {
        if ( fork()== 0 )
            printf("Hi, I am child. My pid is %d\n", getpid());
    }
}
```



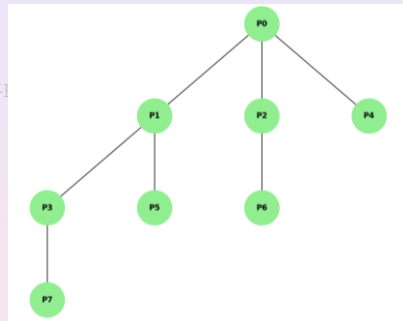
What is the output?

P0: `q2.c`

```
int main() {
    // P1
    printf("before fork, my pid is %d\n", getpid());
    fork(); // P3
    fork(); // P5
    fork(); // P2
    fork(); // P6
    printf("done, my pid is %d\n", getpid());
    // P4
}
```

//q2.c

```
int main() {
    int i;
    printf("before fork, my pid is %d\n", getpid());
    for (i=0; i<3; i++) {
        if (fork() == 0)
            printf("Hi, I am child. My pid is %d\n", getpid());
    }
}
```



Creating a new Process: `fork()`Terminating a process: `exit()``wait()` and `waitpid()`

What is the output?

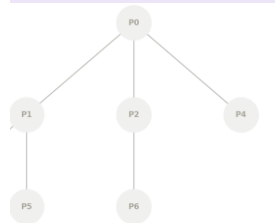
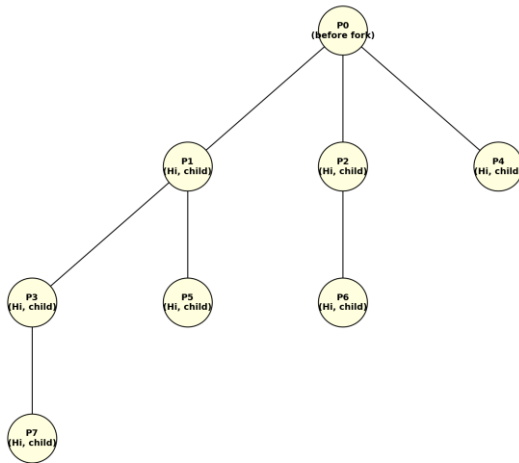
prog.c

```
int main()
{
    printf("P0\n");
    fork();
    fork();
    fork();
    printf("P4\n");
}
```

prog.c

```
int main()
{
    int i;
    printf("P0\n");
    for (i=0; i<10; i++)
    {
        if (i%2 == 0)
            printf("P4\n");
    }
}
```

Process Tree with Messages



Part IV

Terminating a process: `exit()`



Process termination: `exit()`

Synopsis: `void exit(int status);`

- This call terminates a process and never returns
- The `status` value is available to the parent process through the `wait()` system call.

When invoked by a process, the `exit()` system call:

1. Closes all the process's file descriptors
2. Frees the memory used by its code, data and stack
3. Sends a **SIGCHLD** *signal* to its parent and
4. Waits for the parent to accept its return code.

Part V

`wait()` and `exit()`



Creating a new Process: `fork()`Terminating a process: `exit()``wait()` and `waitpid()`

`wait()` and `waitpid()`

`wait()`

Synopsis: `pid_t wait(int *status);`

Allows the parent to wait for the termination of one of its children and accept its termination code.

`waitpid()`

Synopsis: `pid_t waitpid(pid_t pid, int *status, int options);`

`waitpid()` is similar to `wait()` except that it has a number of options that control which process it waits for.

For example:

- If `pid > 0` then, `waitpid()` waits for the child process identified by `pid`.
- If `pid = -1` then, `waitpid()` waits for any child process to return.

Part VI

Assignment #1: Overview



Assignment #1: Overview

- It's all about Coding!
- Read your examples
- Access your textbook
- Work methodology:
 - Start as early as possible
 - Read carefully!
 - Read again!
 - Reach out for clarifications early
 - Make your example easily readable
- Submit way before the deadline
- Avoid using the grace period



Part VII

Attendance Quiz (Not graded)



Quiz



Attendance Quiz

701968234901768023405879312016
670980321768767



774862586034187069210783468019
701968234901768023405879312016
670980321768767

WPOYFXI LFJXI G I E
H I MPHLOM ZFOLI PG
BXNYSXWHYGI JLG I L
OLI EPC OOUJPL QDZC
NYDWIGL OUPJL QDZC

Thank you!
Questions?

DEFYXHQGRUOSGRP RN	3236343972474116
QJRMVQJXVRI DKUEVP	893833239594578
KGWBGQFHP RSUP RU	0310019701552750
XURNYUXZ DCSYUUM I	7875465400196596
WHMFSP UIQYSUQYRSHRG	5019353606937419

00110011100111
11111010000100
11000000001111
00111100111100
00101111111100
00000111011011
11001100111100

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor